

```

"""
Spam Detector – complete pipeline
-----
Requirements: pip install pandas scikit-learn requests
Run: python spam_detector.py
"""

import re
import os
import requests
import zipfile
import io
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
import pickle

```

✓ Core Libraries and Imports

This section imports all the necessary Python libraries required for the spam detection pipeline. These include libraries for regular expressions, file operations, web requests, data manipulation (pandas), machine learning (scikit-learn for model selection, feature extraction, classification, and metrics), and model persistence (pickle).

```

import os
import pandas as pd
import re # Added to ensure 're' is available for clean_text function
import requests # Added for data download logic
import zipfile # Added for data download logic
import io # Added for data download logic
from sklearn.model_selection import train_test_split # Added for train/test split
from sklearn.feature_extraction.text import TfidfVectorizer # Added for feature ex
from sklearn.linear_model import LogisticRegression # Added for model training
from sklearn.naive_bayes import MultinomialNB # Added for model training
from sklearn.metrics import classification_report, confusion_matrix # Added for mo
import pickle # Added for model saving/loading
import matplotlib.pyplot as plt # Added for plotting
import seaborn as sns # Added for plotting

# Section: Data Loading and Initial Preparation

def load_data(csv_path: str, text_column: str, label_column: str):
    """Load data from a CSV file."""
    if not os.path.exists(csv_path):
        print(f"Error: CSV file not found at {csv_path}")
        return None

    df = pd.read_csv(csv_path)
    print(f"Loaded {len(df)} messages from {csv_path}")

```

```

# Ensure the columns are named 'label' and 'text' for consistency
# with the rest of the pipeline.
df = df.rename(columns={label_column: "label", text_column: "text"})

# Convert labels to lowercase to handle potential case inconsistencies
df['label'] = df['label'].str.lower()

# This assumes 'spam' is one of the unique labels in your dataset.
# Adjust if your 'positive' class label is different (e.g., 'malicious', 'offe
df["label_num"] = (df["label"] == "spam").astype(int)

print(f"  Spam: {df['label_num'].sum()} | Ham: {(df['label_num']==0).sum()}")
return df

# Section: Text Preprocessing / Data Manipulation

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " url ", text)
    text = re.sub(r"\d+", " num ", text)
    text = re.sub(r"^\w\s", " ", text)
    return text.strip()

# Section: Model Training and Evaluation

def train_and_evaluate(df):
    df["clean"] = df["text"].apply(clean_text)

    X_train, X_test, y_train, y_test = train_test_split(
        df["clean"], df["label_num"], test_size=0.2, random_state=42, stratify=df[
    ])

    vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, C=1.0),
        "Naive Bayes":          MultinomialNB(alpha=0.1),
    }

    best_model, best_f1 = None, 0
    for name, model in models.items():
        model.fit(X_train_vec, y_train)
        preds = model.predict(X_test_vec)
        report = classification_report(y_test, preds, output_dict=True)
        # The f1 score for 'spam' class (label 1)
        f1 = report["1"]["f1-score"]

        print(f"\n{'-'*40}")
        print(f"  {name}")
        print(f"{'-'*40}")
        print(classification_report(y_test, preds, target_names=["ham", "spam"]))
        cm = confusion_matrix(y_test, preds)
        print(f"  Confusion matrix:\n      TN={cm[0,0]}  FP={cm[0,1]}\n      FN={cm[1,1]

```

```

# --- Add plotting for each model ---
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle(f'Evaluation for {name}', fontsize=16)

# Plot Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['ham', 'spam'], yticklabels=['ham', 'spam'])
axes[0].set_title('Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('True')

# Plot Classification Report (F1-score, Precision, Recall)
metrics_df = pd.DataFrame(report).transpose()
# Access class-specific metrics using numerical labels '0' and '1'
metrics_to_plot = metrics_df[['precision', 'recall', 'f1-score']].loc[['0',
# Rename the index for better plotting labels
metrics_to_plot = metrics_to_plot.rename(index={'0': 'ham', '1': 'spam'})
metrics_to_plot.plot(kind='bar', ax=axes[1], rot=0)
axes[1].set_title('Classification Report Metrics')
axes[1].set_ylabel('Score')
axes[1].set_ylim(0, 1.1)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
# --- End plotting for each model ---

if f1 > best_f1:
    best_f1, best_model = f1, (name, model)

print(f"\n Best model: {best_model[0]} (spam F1 = {best_f1:.3f})")
return best_model[1], vectorizer

# Section: Model Saving / Persistence

def save_model(model, vectorizer, path="spam_model.pkl"):
    with open(path, "wb") as f:
        pickle.dump({"model": model, "vectorizer": vectorizer}, f)
    print(f"\n Model saved to {path}")

# Section: Prediction Function

def load_and_predict(texts, model_path="spam_model.pkl"):
    """Load saved model and classify a list of messages."""
    with open(model_path, "rb") as f:
        bundle = pickle.load(f)
        model = bundle["model"]
        vectorizer = bundle["vectorizer"]

    cleaned = [clean_text(t) for t in texts]
    vecs = vectorizer.transform(cleaned)
    preds = model.predict(vecs)
    probs = model.predict_proba(vecs)[: , 1]

    print("\n— Predictions —————")
    for text, pred, prob in zip(texts, preds, probs):

```

```

label = "SPAM" if pred == 1 else "ham "
bar = "█" * int(prob * 20) + "░" * (20 - int(prob * 20))
print(f" [{label}] {prob:.0%} [{bar}] {text[:60]}")

# Section: Main Execution Block

if __name__ == "__main__":
    print("\n=== Spam Detector ===\n")

    # For the original dataset (if it's already downloaded or locally available as
    # df = load_data(csv_path="spam.csv", text_column="text", label_column="label")

    # Example of how to use with a new CSV file. Replace with your actual path and
    # For instance, if you have a CSV named 'my_data.csv' with columns 'message_co
    # df = load_data(csv_path="my_data.csv", text_column="message_content", label_

    # Currently using the original dataset as an example since a new one hasn't be
    # You'll need to remove the original download logic if you only want to use lo

    # Original logic to download and process SMS spam collection

    local_path = "spam.csv"
    if not os.path.exists(local_path):
        print("Downloading dataset...")
        url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00228/sms
        r = requests.get(url, timeout=15)
        z = zipfile.ZipFile(io.BytesIO(r.content))
        with z.open("SMSSpamCollection") as f:
            lines = f.read().decode("utf-8").strip().split("\n")

        rows = [line.split("\t", 1) for line in lines]
        original_df = pd.DataFrame(rows, columns=["label", "text"])
        original_df.to_csv(local_path, index=False)
        print(f" Saved {len(original_df)} messages to {local_path}")

    # Load the (potentially downloaded) spam.csv using the new load_data function
    df = load_data(csv_path=local_path, text_column="text", label_column="label")

    if df is not None:
        model, vectorizer = train_and_evaluate(df)
        save_model(model, vectorizer)

        # Test on fresh examples
        test_messages = [
            "WINNER!! You have been selected for a cash prize. Call now!",
            "Hey, are we still on for lunch tomorrow?",
            "FREE entry to win £1000 gift card. Text WIN to 80085",
            "Can you send me the report before the meeting?",
            "Congrats! ur mobile has won our £2000 prize. Claim at www.prize4u.com
            "I'll be home by 7, can you pick up some groceries?",
        ]
        load_and_predict(test_messages)

    print("\nDone. To classify your own messages, call load_and_predict(['your

```


=== Spam Detector ===

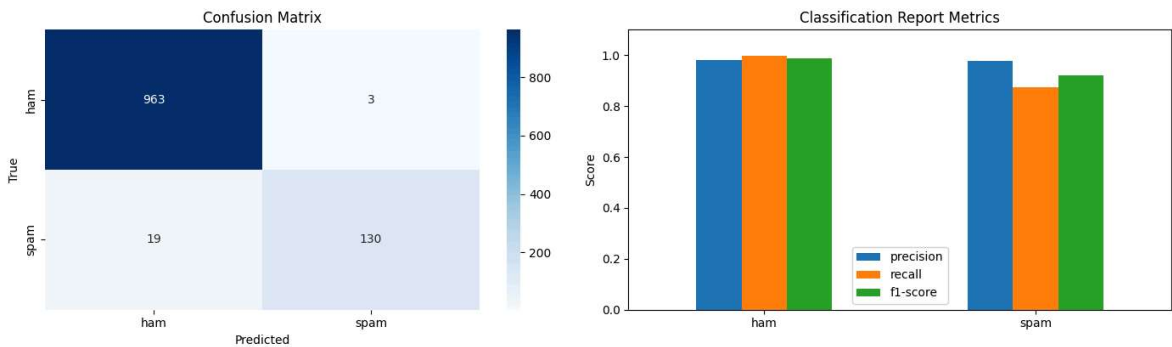
Loaded 5574 messages from spam.csv
Spam: 747 | Ham: 4827

Logistic Regression

	precision	recall	f1-score	support
ham	0.98	1.00	0.99	966
spam	0.98	0.87	0.92	149
accuracy			0.98	1115
macro avg	0.98	0.93	0.96	1115
weighted avg	0.98	0.98	0.98	1115

Confusion matrix:
TN=963 FP=3
FN=19 TP=130

Evaluation for Logistic Regression

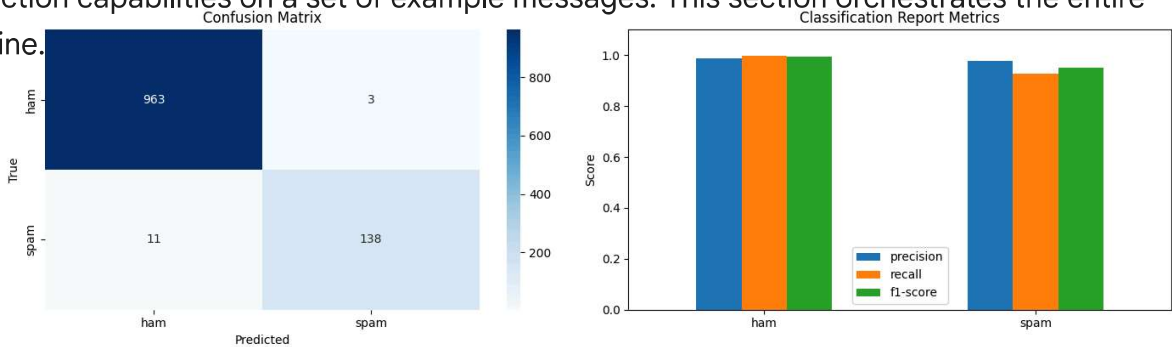


Naïve Bayes

	precision	recall	f1-score	support
ham	0.99	1.00	0.99	966
spam	0.98	0.93	0.95	149
accuracy			0.99	1115
macro avg	0.99	0.99	0.99	1115
weighted avg	0.99	0.99	0.99	1115

Confusion matrix:
TN=963 FP=3
FN=11 TP=138

Evaluation for Naïve Bayes



Best model: Naïve Bayes (spam F1 = 0.952)

6. Main Execution Block (if __name__ == "__main__":)

This block defines the main flow of the spam detection application when the script is run directly. It handles the initial data acquisition (downloading the dataset if not present), loads the data, trains and evaluates the models, saves the best model, and then demonstrates its prediction capabilities on a set of example messages. This section orchestrates the entire pipeline.

5. Prediction Function (`load_and_predict`)

— Predictions
This function provides a simple interface for using the saved model for making predictions on new, unseen messages. It loads the previously saved model and vectorizer, cleans and vectorizes the input texts, and then predicts whether each message is 'spam' or 'ham' along with a probability score. It outputs the results in a concise, friendly format.

```
[spam] 100% | WINNER!! You have been selected for a cash, pr
[ham] 0% | Have you still got free lunch tomorrow?
[spam] 100% | FREE entry to win £1000 gift card. Text WIN t
[ham] 0% | Can you send me the report before the meeting?
[spam] 100% | This is a scam. Please do not provide any info
[ham] 0% | I'll be home by 7, can you pick up some groceri
```

4. Model Saving Function (`save_model`)

This function serializes and saves the trained machine learning model and the TF-IDF vectorizer to a file using `pickle`. This allows the trained pipeline components to be reloaded later for making predictions without needing to retrain the models.

3. Model Training and Evaluation Function (`train_and_evaluate`)

This core function takes the processed DataFrame, cleans the text, and splits the data into training and testing sets. It then vectorizes the text using TF-IDF and trains two different classification models: Logistic Regression and Naive Bayes. After training, it evaluates each model's performance using a classification report and confusion matrix, selecting and returning the model with the best F1-score for spam detection.

T B I <> ↺ 🖼️ “ 1/3 ⋮ — Ψ 😊 ☰ Close

```
### 2. Text Cleaning Function (`clean_text`)
This utility function preprocesses raw text
lowercase, replaces URLs and numbers with ge
punctuation. This standardization helps in r
effectiveness of text feature extraction.
```

2. Text Cleaning Function

(`clean_text`)

This utility function preprocesses raw text messages. It converts text to lowercase, replaces URLs and numbers with generic tokens, and removes punctuation. This standardization helps in reducing noise and improving the effectiveness of text feature extraction.

1. Data Loading Function (`load_data`)

This function is responsible for loading the dataset from a specified CSV file. It ensures the file exists, reads it into a pandas DataFrame, renames columns for consistency, converts labels to a standardized format, and creates a numerical representation for the labels. It also provides a summary of the loaded data, including the count of spam and ham messages.

```
from __main__ import load_and_predict
```